EXPERIMENT 3

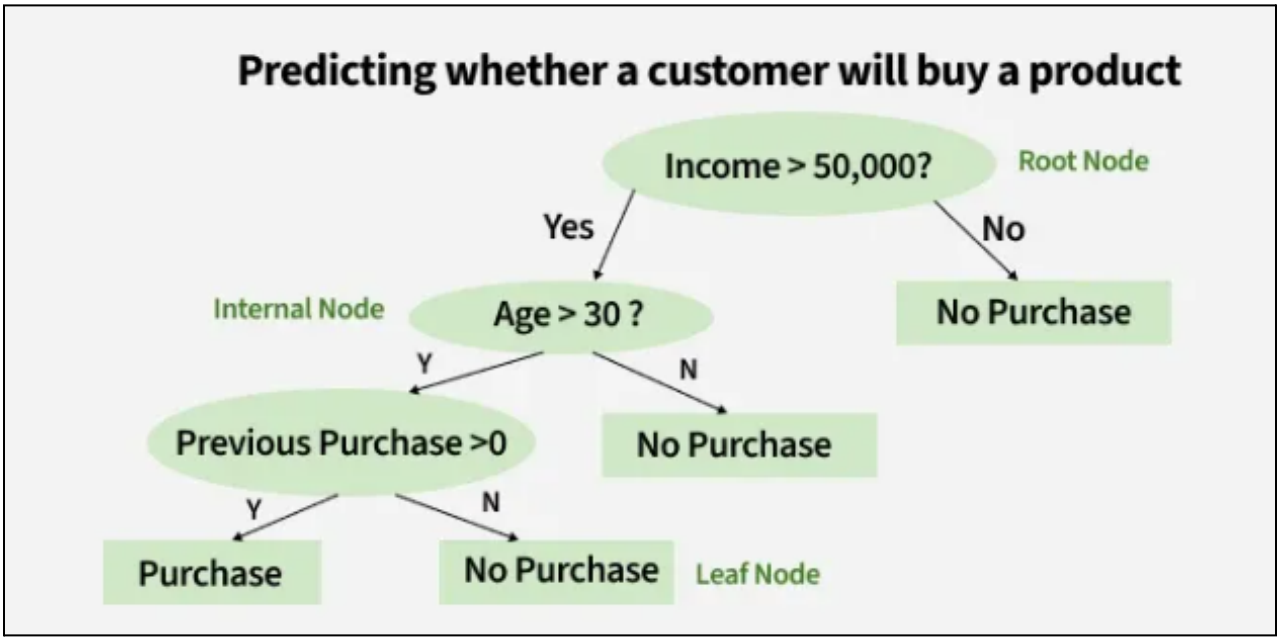
Apply Decision Tree and Random Forest for Classification Tasks

Decision Tree for Classification

A decision tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure which consists of a root node, branches, internal nodes and leaf nodes. It works like a flowchart help to make decisions step by step where:

- Internal nodes represent attribute tests
- Branches represent attribute values
- Leaf nodes represent final decisions or predictions.

How Does a Decision Tree Work :
A decision tree splits the dataset based on feature values to create pure subsets ideally all items in a group belong to the same class. Each leaf node of the tree corresponds to a class label and the internal nodes are feature-based decision points. Let's understand this with an example



Working Principle

A Decision Tree splits the dataset into subsets based on feature values. At each step, the algorithm selects the best feature to split the data such that the resulting subsets are as pure as possible.

Purity is measured using:

1. Gini Index

$$Gini = 1 - \sum p_i^2$$

Where:

- pip_ipi is the probability of class i.

Lower Gini value indicates better split (more pure node).

2. Entropy

$$Entropy = -\sum p_i \log_2(p_i)$$

Lower entropy means higher purity.

The algorithm selects the split that provides the highest Information Gain.

Advantages

- Easy to understand and interpret
- Requires minimal data preprocessing
- Works well with both numerical and categorical data

Random Forest for Classification

Introduction

Random Forest is an ensemble learning algorithm that improves the performance of Decision Trees by combining multiple trees.

Instead of building a single tree, Random Forest builds many trees and combines their outputs using majority voting.

Working Principle

Random Forest follows these steps:

1. Randomly select samples from the dataset (Bootstrap Sampling).
2. Build multiple Decision Trees on different random subsets.
3. At each split, choose random subsets of features.
4. Each tree makes a prediction.
5. Final prediction is made using majority voting.

Mathematically :

$$Final\ Prediction = Mode(Tree_1, Tree_2, ..., Tree_n)$$

Example

If 5 trees predict :

- Tree 1 → Yes
- Tree 2 → No
- Tree 3 → Yes
- Tree 4 → Yes
- Tree 5 → No

Final Output = Yes (Majority Vote)

Advantages

- Reduces overfitting
- Higher accuracy than a single Decision Tree
- Handles large datasets effectively
- More stable and robust

1.Dataset Source

Dataset Name: Titanic – Machine Learning from Disaster
Source: Kaggle
Link:
https://www.kaggle.com/datasets/yasserh/titanic-dataset

File Used: Titanic-Dataset.csv

2.Dataset Description

Problem Type:

Binary Classification

Objective:

Predict whether a passenger survived or not.

Dataset Size:

- Rows: ~891
- Columns: 12

Features Description

Feature	Description
PassengerId	Unique ID
Survived	Target variable (0 = No, 1 = Yes)
Pclass	Ticket class (1,2,3)
Name	Passenger name
Sex	Male/Female
Age	Age
SibSp	Siblings/Spouses aboard
Parch	Parents/Children aboard
Ticket	Ticket number
Fare	Ticket fare
Cabin	Cabin number
Embarked	Port of Embarkation

Target Variable:

Survived

- 0 → Did Not Survive
- 1 → Survived

This is a **binary classification problem**.

3.Mathematical Formulation

Decision Tree

Uses impurity measures:
Gini Index:

$$Gini = 1 - \sum p_i^2$$

Entropy:

$$Entropy = -\sum p_i log_2(p_i)$$

Split chosen using Information Gain :

$$IG = Entropy(parent) - Weighted\ Entropy(children)$$

Random Forest

Random Forest = Multiple Decision Trees
Each tree:

- Trained on bootstrap sample
- Uses random feature subset

Final prediction:

Final Prediction=MajorityVotingFinal\ Prediction = Majority VotingFinal Prediction=MajorityVoting

Reduces:
✔ Overfitting
✔ Variance

4.Algorithm Limitations

1.Decision Tree
Overfits easily
High variance
Unstable to small changes

2.Random Forest
Slower
Less interpretable
High memory usage

5.Methodology / Workflow

Dataset Loading
↓
Data Cleaning
↓
Encoding Categorical Data
↓
Train-Test Split
↓
Model Training
↓
Prediction
↓
Evaluation
↓
Hyperparameter Tuning

6.Performance Analysis Metrics

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

7.Hyperparameter Tuning

Hyperparameter tuning is the process of optimizing model parameters to improve performance. Unlike model parameters, hyperparameters are set before training and control the learning process.

Hyperparameter Tuning for Decision Tree

Important hyperparameters:

- max_depth → Maximum depth of the tree
- min_samples_split → Minimum number of samples required to split a node
- min_samples_leaf → Minimum samples required at a leaf node
- criterion → Gini or Entropy

By tuning these parameters:

- Overfitting can be reduced
- Model generalization improves

Grid Search with cross-validation is commonly used to find the best combination.

Hyperparameter Tuning for Random Forest

Important hyperparameters:

- n_estimators → Number of trees in the forest
- max_depth → Maximum depth of each tree
- min_samples_split → Minimum samples required to split
- max_features → Number of features considered at each split

Increasing n_estimators usually improves performance but increases computational cost.

Method Used: Grid Search Cross-Validation

GridSearchCV performs:

1. Tries multiple combinations of hyperparameters.
2. Uses cross-validation (e.g., 5-fold CV).
3. Selects the combination that gives the best validation accuracy.

Outcome of Hyperparameter Tuning

After tuning:

- The optimized Decision Tree shows reduced overfitting.
- The optimized Random Forest achieves improved accuracy.
- Random Forest typically remains the best-performing model.